

AURA-X

Autonomous Unified Reliability & Evolution Analyzer

Project Status & Remaining Plan

GitHub Repository Integration & Ingestion System

4 of 16 phases complete — 57/57 backend tests passing

Document: docs/AURA-X_PROJECT_STATUS_AND_PLAN.pdf

Supersedes progress tracking in docs/GITHUB_INTEGRATION_PLAN.pdf (original plan, unchanged)

Phase Status Overview

#	Phase	Status	Notes
0	Bootstrap (backend + frontend skeleton)	DONE	FastAPI + React/Vite scaffolds, wired, tested
1	Audit existing project	DONE	docs/github_integration_audit.md
2	Design integration architecture	DONE	docs/GITHUB_INTEGRATION.md architecture section
3	RepositoryProvider abstraction	DONE	app/domain/ layer, 13 tests
4	GitHub URL parsing & validation	DONE	app/domain/github_url.py, 34 tests
5	GitHub API client	PENDING	Real HTTP calls begin here
6	Metadata & branch retrieval	PENDING	
7	Secure repository cloning	PENDING	
8	RepositoryContext construction (scan/lang/evolution)	PENDING	
9	Database persistence	PENDING	Needs Postgres provisioned + Alembic init
10	REST API layer	PENDING	
11	Async status tracking	PENDING	
12	Downstream analysis hookup	PENDING	
13	Dashboard integration	PENDING	
14	Excel reporting	PENDING	
15	Full test suite (security/perf)	PENDING	
16	End-to-end validation	PENDING	

PHASE 0 | Bootstrap

✓ DONE

What was built

Stood up a real, runnable FastAPI backend and React/Vite/TypeScript frontend, wired to each other, before any GitHub-specific code existed.

Implementation Detail

- backend/app/main.py, app/api/v1/router.py (health check), app/core/config.py (Settings incl. GITHUB_TOKEN as SecretStr), app/core/logging.py.
- backend/app/db/base.py + session.py (SQLAlchemy engine/session, no tables yet).
- frontend/src/App.tsx replaced with a connectivity smoke test hitting /api/v1/health.
- Git repository initialized; .gitignore excludes .venv/node_modules/.env/.workspace.

Deliverables

- backend/, frontend/, README.md, .gitignore, first commit d13bf45.

Verified

Backend boots and responds to curl; frontend npm run build succeeds; 3/3 tests passing.

PHASE 1 | Audit Existing Project

✓ DONE

What was built

Documented the Phase 0 state before Phase 2+ began: what's reusable, what's missing, integration boundaries, and risks.

Implementation Detail

- Catalogued every backend/frontend file and its role.
- Identified reusable pieces: Settings, get_db(), api_router, CORS config, test client fixture.
- Flagged risks: no Postgres provisioned yet, no Alembic env initialized, clone-library choice (gitpython vs raw subprocess) deferred to Phase 7.

Deliverables

- docs/github_integration_audit.md

Verified

Audit reviewed; baseline 3/3 tests green; no migration needed (clean slate).

PHASE 2 | Design Integration Architecture

✓ DONE

What was built

Defined the RepositoryProvider/RepositoryContext boundary so no downstream module ever imports a GitHub-specific type.

Implementation Detail

- RepositoryProvider interface table (fetch_metadata, list_branches, get_commit_history, get_languages, clone).

- RepositoryContext field table matching the plan's spec exactly.
- Ingestion state diagram: PENDING -> VALIDATING -> FETCHING_METADATA -> FETCHING_BRANCHES -> CLONING -> SCANNING -> READY, with FAILED reachable from any non-terminal state.

Deliverables

- docs/GITHUB_INTEGRATION.md (architecture section).

Verified

Design reviewed and used verbatim as the spec for Phase 3's implementation.

PHASE 3 | RepositoryProvider Abstraction

✓ DONE

What was built

Implemented the provider-agnostic domain layer with zero external dependencies (no HTTP client, no ORM, no GitHub SDK) so it is fully unit-testable offline.

Implementation Detail

- app/domain/models.py -- RepositoryMetadata, BranchInfo, CommitInfo, FileEntry, CloneResult (frozen dataclasses).
- app/domain/errors.py -- structured error taxonomy (INVALID_REPOSITORY_URL, REPOSITORY_NOT_FOUND, RATE_LIMITED, CLONE_FAILED, INVALID_STATE_TRANSITION, ...).
- app/domain/repository_context.py -- RepositoryContext with an enforced state machine (transition_to() raises InvalidStateTransitionError on an illegal jump; terminal states have no outgoing edges).
- app/domain/repository_provider.py -- RepositoryProvider ABC + hostname-keyed provider registry (register_provider / get_provider_class_for_host).

Deliverables

- app/domain/ package; tests/test_repository_context.py, tests/test_repository_provider.py (13 tests).

Verified

16/16 tests passing (3 baseline + 13 new); no network or database dependency in this layer.

PHASE 4 | GitHub URL Parsing and Validation

✓ DONE

What was built

First code that touches real (untrusted) user input. Normalizes a GitHub URL into (owner, repo) or rejects it with a structured, actionable error before anything downstream runs.

Implementation Detail

- app/domain/github_url.py::parse_github_url() -- accepts https/http, .git suffix, trailing slash, www. host.
- Enforces real GitHub owner/repo naming rules (owner: alnum/hyphen, no leading/trailing hyphen, <=39 chars; repo: alnum/./_/-, <=100 chars, rejects '.', '..', '.git').
- Rejects: wrong scheme, missing host, embedded credentials, path traversal ('..', '%2e%2e', backslashes), control/CRLF characters, lookalike hosts (github.evil.com, github.com.evil.com), overlong input (>2048 chars).

Deliverables

- app/domain/github_url.py; tests/test_github_url.py (34 tests incl. a security matrix).

Verified

57/57 tests passing overall. Every rejection path returns `InvalidRepositoryUrlError` or `UnsupportedRepositoryProviderError`, never a raw exception.

Remaining Phases (5-16)

Unchanged in substance from the original plan; reproduced here at slightly condensed length now that the domain layer they build on already exists as real, tested code (app/domain/*).

PHASE 5 | GitHub API Client

PENDING

Goal

Centralize every GitHub HTTP call behind one client (app/services/github_client.py) implementing RepositoryProvider's data-fetch methods against the real GitHub REST API.

Key Tasks

- GET /repos/{owner}/{repo}, /branches, /commits, /commits/{sha}, /languages via httpx.
- Timeouts (github_request_timeout_seconds), bounded retries (github_max_retries), pagination.
- Rate-limit detection (X-RateLimit-* headers) -> RateLimitExceededError.
- Optional Authorization header from settings.github_token; never logged.
- Wrap all raw httpx/GitHub exceptions into app.domain.errors types before they leave the client.

Deliverables

- app/services/github_client.py; GitHubProvider registered for github.com.

Exit Criteria

Mocked-response tests (respx) cover 200/404/401/403/429/timeout/malformed/pagination; zero raw GitHub HTTP calls exist outside this module.

PHASE 6 | Metadata and Branch Retrieval

PENDING

Goal

Use GitHubProvider to populate real repository metadata, branch list, default-branch resolution, and bounded commit history for a real public repository.

Key Tasks

- GitHubProvider.fetch_metadata/list_branches/get_commit_history implemented against the live API.
- Default-branch fallback when caller omits a branch; BranchNotFoundError when an invalid branch is requested.

Deliverables

- Integration-style tests against a small real public repo (network-gated, skippable).

Exit Criteria

A real public repository URL returns accurate metadata/branches/history end-to-end.

PHASE 7 | Secure Repository Cloning

PENDING

Goal

Clone real source into workspace_root/{repository_id}/source using argument-list subprocess calls only, with timeout and size limits from Settings.

Key Tasks

- Isolated per-repository workspace directory; no execution of repository code (no setup.py, no install scripts).
- Enforce clone_timeout_seconds and max_repository_size_mb; verify HEAD/branch/non-empty post-clone.
- Safe cleanup on failure; path-traversal-proof workspace path construction.

Deliverables

- app/services/clone_service.py.

Exit Criteria

Shell-injection and oversized-repo security tests pass; commit SHA is verifiably correct for the checked-out branch.

PHASE 8 | RepositoryContext Construction

PENDING

Goal

Turn API metadata + the cloned tree into a fully populated RepositoryContext: file inventory, language detection, test-framework detection, and Git evolution signals (churn, co-change).

Key Tasks

- File scan respecting .gitignore, excluding .git/node_modules/venvs/caches/binaries/oversized files.
- Language detection via extension + GitHub languages + dependency files.
- Test framework detection (pytest/unittest/tox/nox, Jest/Vitest/Mocha) -- no command execution.
- Churn/co-change calculation from git_history feeding the future Risk Engine.

Deliverables

- Populated RepositoryContext for a real repo; unit tests against fixture commit history.

Exit Criteria

RepositoryContext for a real public repo has non-empty file_tree, languages, and evolution signals.

PHASE 9 | Database Persistence

PENDING

Goal

Persist Repository/Branch/AnalysisRun using the existing SQLAlchemy Base and the state machine already implemented in Phase 3.

Key Tasks

- Provision Postgres (or SQLite for early dev) -- resolves the risk flagged in the Phase 1 audit.
- alembic init; first migration for Repository/Branch/AnalysisRun tables.
- Persist structured errors against AnalysisRun; enforce the same ALLOWED_TRANSITIONS at the DB layer.

Deliverables

- app/models/*, alembic/ migrations.

Exit Criteria

A completed ingestion is fully persisted and re-readable with full repository/branch/commit/run lineage.

PHASE 10 | REST API Layer

PENDING

Goal

Extend the existing `api_router` (`app/api/v1/router.py`) with the ingestion and browsing endpoints.

Key Tasks

- POST `/api/v1/repositories/github`, GET `/repositories`, GET `/repositories/{id}`, `/branches`, `/profile`, `/commits`, POST `/repositories/{id}/refresh`.
- Pydantic request/response models; structured error responses; pagination.

Deliverables

- `app/api/v1/repositories.py`.

Exit Criteria

Full API test suite passes: valid ingestion, invalid URL, branch selection, profile, refresh, status polling, pagination.

PHASE 11 | Asynchronous Status Tracking

PENDING

Goal

Background execution of ingestion so requests don't block, with status reflecting real stage transitions (reusing `IngestionStatus` from Phase 3).

Key Tasks

- Background worker/task wired to the Phase 9 state machine.
- Status endpoint reflects live state; failures surface FAILED with the structured error.

Deliverables

- Ingestion orchestration service.

Exit Criteria

Frontend can poll real, monotonically progressing status for an in-flight ingestion.

PHASE 12 | Downstream Analysis Hookup

PENDING

Goal

Feed the completed `RepositoryContext` into Repository Intelligence / Evolution / Risk / Test Planning without re-deriving repository facts.

Key Tasks

- Downstream entry points accept `RepositoryContext` as sole repository input.
- Integration test: one commit SHA flows unchanged through every downstream stage.

Deliverables

- Downstream module wiring.

Exit Criteria

Single ingestion run produces one RepositoryContext consumed identically by every stage.

PHASE 13 | Dashboard Integration

PENDING

Goal

Replace the Phase 0 connectivity-check UI with the real onboarding flow: paste URL, pick branch, watch real progress, view the profile.

Key Tasks

- Repository onboarding page wired to Phase 10 endpoints.
- Real progress bar driven by the Phase 11 status endpoint, no simulated progress.

Deliverables

- frontend onboarding screen/components.

Exit Criteria

Full onboarding flow completed in-browser using only real API data.

PHASE 14 | Excel Reporting

PENDING

Goal

Route repository facts into Excel export using the same models as the API/dashboard.

Key Tasks

- Repository URL, provider, owner, name, branch, commit SHA, languages, timestamp added to sheet.
- Snapshot test asserting no secret-like values in generated workbook.

Deliverables

- Excel export update.

Exit Criteria

Generated Excel for a completed run has accurate repo facts, zero credential leakage.

PHASE 15 | Comprehensive Test Suite

PENDING

Goal

Reach full coverage: unit, API-client, git-integration, API, and security dimensions; offline-safe by default with an opt-in network tier.

Key Tasks

- Git integration tests against temporary local repos.

- Full security matrix: injection, traversal, oversized repos, token-leakage across logs/errors/responses/reports.

Deliverables

- CI-runnable full suite.

Exit Criteria

Entire suite green offline; network-tier green when enabled; no credentials in any output.

PHASE 16 | End-to-End Validation

PENDING

Goal

Prove the complete pipeline for a real user with a real public repository, exactly as specified in the original plan's completion requirements.

Key Tasks

- Full walk: URL -> validate -> metadata/branches -> clone -> commit SHA -> scan -> RepositoryContext -> persist -> API -> dashboard -> downstream analysis.
- Finalize docs/GITHUB_INTEGRATION.md and README with real usage instructions.

Deliverables

- Signed-off demo run; finalized docs.

Exit Criteria

A real public GitHub repository genuinely enters and completes the AURA-X pipeline end-to-end with no mocked step.

This document tracks living project status. Regenerate it (python docs/generate_project_status_plan_pdf.py) after completing each further phase.